

Vue中v-if和v-show条件渲染指令的全面解析

在 Vue 中，`v-if` 和 `v-show` 都是用于条件渲染的核心指令，但底层实现、性能表现、适用场景截然不同。下面从核心原理、语法特性、性能对比、适用场景四个维度全面解析，帮你精准选择。

一、核心原理（最本质区别）

指令	底层实现方式	DOM 存在性
<code>v-if</code>	动态创建/销毁 DOM 元素	条件不满足时，元素从 DOM 中移除
<code>v-show</code>	动态切换 CSS 的 <code>display: none</code> 属性	元素始终存在于 DOM 中，仅隐藏

二、语法特性对比

1. `v-if`：条件性渲染（真正的“渲染/不渲染”）

`v-if` 是惰性的：初始条件不满足时，不会渲染元素（节省初始渲染开销）；条件切换时，会触发元素的创建/销毁，以及组件的生命周期（如 `created` / `destroyed`）。

语法支持：

- 支持 `v-else-if` / `v-else` 多分支判断；
- 可作用于 `<template>` 标签（批量控制多个元素，且 `<template>` 不会生成额外 DOM）；
- 可结合 `key` 优化复用逻辑（避免 Vue 复用相同模板的元素）。

示例：

Code block

```
1  <template>
2    <div>
3      <!-- 基础用法 -->
4      <div v-if="status === 'active'">激活状态</div>
5      <!-- 多分支 -->
6      <div v-else-if="status === 'pending'">待处理</div>
7      <div v-else>已关闭</div>
8
9      <!-- 批量控制（template 无额外 DOM） -->
10     <template v-if="showList">
```

```

11      <li>列表项1</li>
12      <li>列表项2</li>
13    </template>
14  </div>
15 </template>
16
17 <script>
18 export default {
19   data() {
20     return {
21       status: 'active',
22       showList: true
23     }
24   }
25 }
26 </script>

```

2. `v-show`：条件性显示（“渲染后隐藏/显示”）

`v-show` 是**非惰性的**：初始条件不满足时，元素仍会渲染到 DOM，仅通过 `display: none` 隐藏；条件切换时，仅修改 CSS 属性，无 DOM 增删。

语法限制：

- 不支持 `v-else-if` / `v-else`；
- 不能作用于 `<template>` 标签（因为 `<template>` 本身不会渲染到 DOM，无法添加 style）；
- 所有支持 `display` 属性的元素都可使用（少数元素如 `<template>` / `<tr>` 除外）。

示例：

Code block

```

1  <template>
2    <div>
3      <div v-show="isShow">仅隐藏，DOM 仍存在</div>
4      <!-- 错误用法：v-show 不能作用于 template -->
5      <!-- <template v-show="isShow">...</template> -->
6    </div>
7  </template>
8
9  <script>
10 export default {
11   data() {
12     return {
13       isShow: false
14     }

```

```
15     }
16   }
17 </script>
18
19 <!-- 渲染后的 DOM 结构 (isShow=false 时) -->
20 <!-- <div style="display: none;">仅隐藏, DOM 仍存在</div> -->
```

三、性能对比（关键选择依据）

场景	<code>v-if</code> 性能表现	<code>v-show</code> 性能表现
初始渲染（条件假）	开销低（不渲染 DOM）	开销高（渲染 DOM 后隐藏）
条件频繁切换	开销高（反复创建/销毁 DOM）	开销低（仅切换 CSS）
条件极少切换	开销低（一次渲染/销毁）	无优势（始终占用 DOM）

四、适用场景（总结）

优先用 `v-if` 的场景：

- 1. 条件**极少切换**（如页面初始化时根据权限渲染、一次性判断）；
- 2. 条件不满足时，**不希望元素出现在 DOM 中**（如敏感数据、无需 SEO 的隐藏内容）；
- 3. 需要结合 `v-else-if` / `v-else` 实现多分支逻辑；
- 4. 控制大量元素的渲染（避免无用 DOM 占用内存）。

优先用 `v-show` 的场景：

- 1. 条件**频繁切换**（如 Tab 切换、按钮显隐、下拉菜单）；
- 2. 初始渲染后，元素需要快速显示/隐藏（无 DOM 重建开销）；
- 3. 简单的显隐控制（无多分支逻辑）。

五、补充注意事项

- 1. `v-if` 优先级高于 `v-for`：若同时使用，Vue 会先判断 `v-if` 再执行 `v-for`（不推荐同时使用，可将 `v-for` 放在外层 `<template>`）；
- 2. `v-show` 不支持 `<template>`，但可包裹在 `<div>` 中实现批量控制；
- 3. 特殊元素兼容：`<tr>` / `<td>` 等元素的 `display` 属性特殊，`v-show` 仍可正常使用（Vue 会适配）；
- 4. 性能优化：频繁切换的复杂组件，可结合 `v-show` + 懒加载（如 `keep-alive`）。

最终口诀

- 少切换，用 `v-if` （省初始渲染）；
- 常切换，用 `v-show` （省切换开销）；
- 多分支，必用 `v-if`；
- 要隐藏，DOM 留，用 `v-show`；DOM 删，用 `v-if`。

（注：文档部分内容可能由 AI 生成）